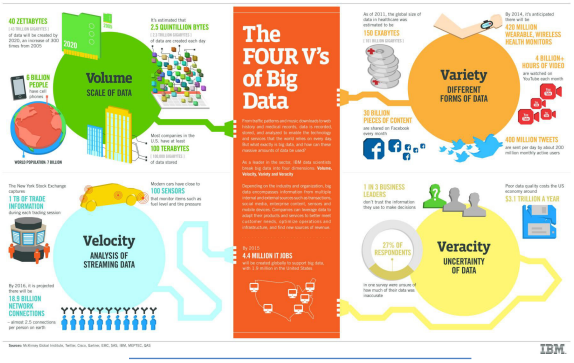
What is Big Data?

- Big Data is high volume, high velocity and high variety of data that require new forms of processing to enable knowledge extraction, insight discovery, decision making, and process optimization

What is Big Data?



What is Big Data?

- The 4 V's of Big Data:

1. **Volume**
   - The main characteristic of Big Data is the volume
   - The volume of data impacts its analysis
   - Historical data is important especially for Business Intelligence
   - Data generated from different sources are stored together to create correlations and extract knowledge.

What is Big Data?

- The 4 V's of Big Data:

2. **Variety**
   - Variety refers to the types of data available for analysis
   - Multiple types of data: numbers, dates, text, images, video, etc.
   - Multiple sources for data: companies databases, social media, blogs, etc.
   - Structured, unstructured and hybrid types of data.

What is Big Data?

- The 4 V's of Big Data:

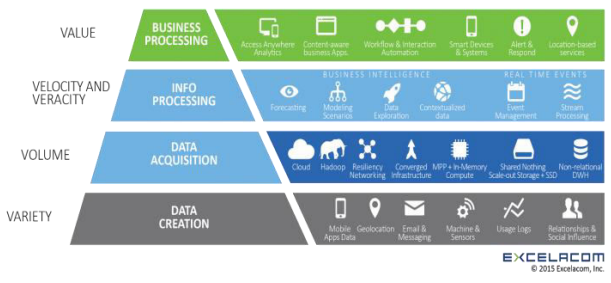
3. **Veracity**
   - Veracity refers to the trustworthiness of the data
   - The quality of the data can affect the analysis process
   - The data must be representative, relevant, consistent, accurate and current to discover patterns
   - The data must be preprocessed to extract relevant knowledge

What is Big Data?

- The 4 V's of Big Data:

4. **Velocity**
   - Velocity refers to how fast the data is processed
   - The faster the date is processed, the faster it can be queried and interpreted to extract knowledge
   - A rapid data analysis helps to correctly and rapidly make and take decision

What is Big Data?



What is Big Data?

- The 8 V's of Big Data:

5. **Value**:
   - A critical objective of Big Data is to use the 4 V to create value
   - Through data analysis, Value is achieved by making the data profitable and help to increase revenue and decrease costs

What is Big Data?

- The 8 V's of Big Data:

6. **Variability** of the data must be also taken into account
   - The meaning of data is constantly changing
   - The context where the data appears must be understood

What is Big Data?

- The 8 V's of Big Data:

7. **Visualization** refers to presentation of data in a pictorial or graphical form
   - Analytics preserved visually
     - To understand difficult concept
     - To identify new patterns
     - Explore the data
     - Detect inconsistencies in the data structure
   - Interactive visualization: charts, graphs, etc.
   - Involves developing new techniques for data visualization

What is Big Data?

- The 8 V's of Big Data:

8. **Vicissitude** refers to the challenges of scaling Big Data complex workflows.
   - MapReduce frameworks
   - Hadoop Ecosystem
   - Spark Environment
   - Blockchain

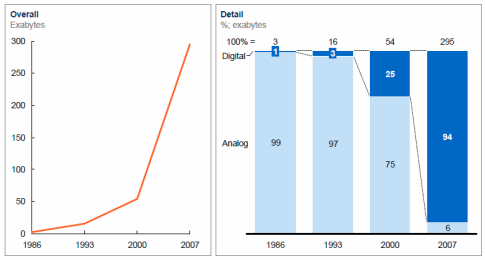
Why Big Data?

- Increased storage capacities
  - Storage capacities have grown
  - Storage (now) is cheap – cost of storage/GB decreased

Why Big Data?

Data storage has grown significantly, shifting markedly from analog to digital after 2000

Global installed, optimally compressed, storage



NOTE: Numbers may not sum due to rounding.
SOURCE: Hilbert and López, "The world's technological capacity to store, communicate, and compute information," *Science*, 2011

Why Big Data?

- Increase of processing power
  - Computation capacity has also risen sharply
  - New means of scaling computations:
    - TPUs (Tensor Processing Units)
    - GPUs (Graphics Processing Units)
    - Grid Computing
    - Cloud Computing
    - Hadoop Ecosystem
    - Spark Environment, etc.

Why Big Data?

- Availability of data
  - More data is available
  - Different types of data are available:
    - Private sector (financial corporations, banks, etc.)
    - Social media
    - Internet of Things

Types of Big Data

- Structured Big Data
- Unstructured Big Data
- Hybrid (Multi-Structured/Semi-Structured) Big data

Types of Big Data

- Structured Big Data
  - Refers to data that have a predefined data model
  - Has a strict schema which generally adheres to the Relational Algebra Theory
  - The data set's attributes are predefined and have a strict data type: number, datetime, string, etc.

Types of Big Data

- Structured Big Data sources:
  - Computer or machine generated data is data that generally refers to data that is created by a machine without human intervention.
  - Some examples:
    - Sensor data: generated by sensors, medical devices, GPS, etc.
    - Web log data: data captured by machine's activates, it is generated by servers, applications, networks, etc.
    - Financial data: financial systems use predefined rules that automate their processes, e.g., stock-trading

Types of Big Data

- Structured Big Data sources:
  - Human generated data is data that is generated by the interaction of human with computers
  - Some examples:
    - Input Data: information and data that humans' input in a computer, e.g., when buying an airplane ticket a human will input his name, age, gender, passport ID, etc.
    - Click-stream data: data that is generated when humans click a link on a website. This data is used to analyze and determine customer behavior and buying patterns
    - Gaming related data: used to record the player's moves in a game to understand and develop new wining strategies

Types of Big Data

- Structured Big Data technologies:
  - Relational databases: Oracle, PostgreSQL, MySQL, Microsoft SQL server, Teradata
  - Data Warehouses and Data Marts
  - Enterprise Resource Planning software (ERP)
  - Customer Relationship Management software (CRM)
  - Mainframes

Types of Big Data

- Unstructured Big Data
  - Refers to data that does not have a predefined data model
  - Does not have a strict schema, generally it uses a flexible schema-free model
  - The data set's attributes
    - Are not predefined
    - They do not have a strict data type
  - Employs the use of different data structures: sets, lists, maps, etc.

Types of Big Data

- Unstructured Big Data sources:
  - As in the case of Structured Big Data, they are machine or human generated
  - Some examples of machine generated unstructured data:
    - Satellite images includes weather data or the data the governments captures in their surveillance satellites
    - Scientific data includes seismic imagery, atmospheric data, etc.
    - Photographs and videos includes security, surveillance and traffic videos
    - Radar and sonar data includes meteorological and oceanographic seismic profiles.

Types of Big Data

- Unstructured Big Data sources:
  - Some examples of human generated unstructured data:
    - Text internal to a company: word documents, excel spreadsheets, emails, etc.
    - Social media data: Facebook, Twitter, YouTube, LinkedIn, Instagram, etc.
    - Mobile data: text messages, location information, etc.
    - Website content: News sites, YouTube, Instagram, etc.

Types of Big Data

- Unstructured Big Data technologies:
  - NoSQL databases: MongoDB, CouchDB, Neo4J, Cassandra, etc.
  - Distributed storage frameworks: Hadoop
  - Data Lakes (a storage repository that stores vast amounts of raw data in its native format).
  - Enterprise Content Management Systems (CMS) that manage the complete life cycle of contend

Types of Big Data

- Hybrid (Multi-structured/Semi-structured) Big Data
  - Combines Structured and Unstructured Big Data
  - Combines the data sources for Structured and Unstructured Big Data and correlates the information
  - Combines the technologies used for structured and unstructured Big Data

Techniques

- When Big data is a problem?

1. When the operations on the data are complex:
   - Modeling and reasoning
   - Knowledge extraction
   - Pattern extraction
2. When usual algorithms cannot handle big amounts of data:
   - Performance and query execution time
   - Parallelism and distribution
   - Resource usage (CPU, RAM)

Techniques

- Big Data combines multiple research fields:
  - Databases
  - Knowledge Discovery in Databases (KDD)
  - Data Mining and Text Mining
  - Parallel and distributed algorithms
  - High Performance Computing (HPC)
  - Information Retrieval (IR) and subdomains: Multi-Lingual Information Retrieval (MLIR)
  - Natural Language Processing (NLP)
  - Semantic Web
  - Probabilities and Statistics
  - Machine Learning
  - Etc.

Distributed architectures

- A distributed architecture is a model in which resources located on network computers are put together in a resource pool to achieve a common goal.
- A cluster is a set of tightly connected computers that work together so that, in many respect they can be seen as a single system

Distributed architectures

- Scalability is the capability of a system to handle a growing amount of work
- Types of scaling
  - Vertical scaling (scale up/down)
    - Means that you can scale by adding more resources to an existing machine, e.g., CPU, RAM, HDD, etc.
  - Horizontal scaling (scale out/in)
    - Means that you can scale by adding more machines into the resource pool, e.g., add nodes to the cluster.

Distributed architectures

- Resource sharing in a cluster
  - Shared everything architecture
    - Is a distributed computer architecture in which each node is added its resources to the resource pool
    - More specifically, the user sees the total amount of memory, CPU or disk storage and not individual amounts
  - Shared nothing architecture
    - Is a distributed computer architecture in which each node is independent and self sufficient
    - More specifically, none of the nodes share memory, CPU or disk storage

Distributed architectures

- Replication involves sharing information to
  - Ensure consistency between redundant resources
  - Improve reliability, fault-tolerance, or accessibility
- Replication is archived when the same data is stored on multiple storage devices

Distributed architectures

- Replication from the architecture perspective
  - Master-Slave (Primary-Secondary)
    - The Master node supports all the operations, i.e., create, read, update, delete (CRUD)
    - Only the Master node can start a transaction
    - Only the read operation is supported on the Slave node
  - Multi-Master (Multi-Primary)
    - All the nodes support all the CRUD operations
    - All the nodes can start a transaction

Distributed architectures

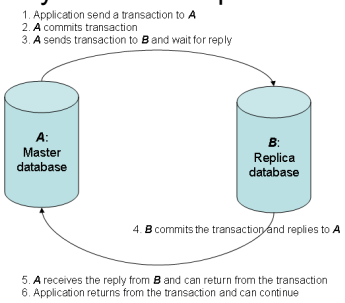
- Types of replication from the transaction perspective
  - Synchronous replication
    - Guarantees 0 data loss, i.e., a transaction either is completed on all the nodes or not at all
    - A transaction is not considered complete without acknowledgment from all the nodes
    - Applications wait for a write transaction to complete before proceeding with further work, hence overall performance decreases considerably.

Distributed architectures

- Types of replication from the transaction perspective
  - Asynchronous replication
    - Transaction is considered complete as soon as the local storage acknowledges it
    - Remote nodes are updated but with (a small) lag
    - Performance is greatly increased, but in case of losing a local storage, the remote storage is not guaranteed to have the current copy of data and most recent data may be lost.

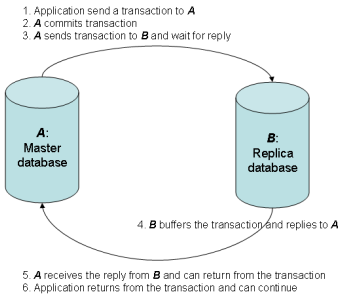
Distributed architectures

Synchronous replication

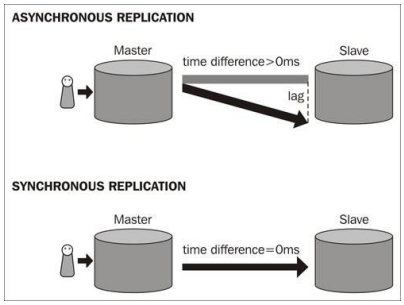


Distributed architectures

Asynchronous replication



Distributed architectures



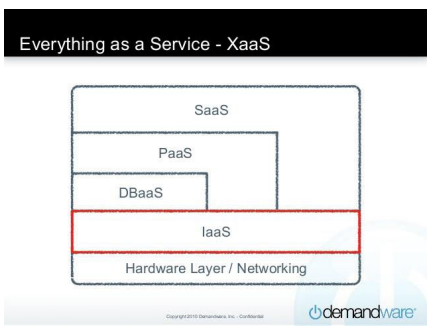
Cloud Computing

- Cloud Computing
  - Is the delivery of on-demand computing resources over the internet on a pay-for-use basis
  - The resources available include everything from data centers to applications
  - Often referred as simply “the cloud”

Cloud Computing

- Cloud Computing
  - Infrastructure as a Service (IaaS)
  - Database as a Service (DBaaS)
  - Platform as a Service (PaaS)
  - Software as a Service (SaaS)

Cloud Computing



Cloud Computing

- IaaS (Cloud Computing)
  - Is an instant computing infrastructure provisioned and managed over the internet.
  - It provides virtual computing resources over the internet

Cloud Computing

- IaaS features:
  - Provision fundamental computing resources in the form of Virtual Machines (VM)
  - Resources are distributed and support dynamic scaling
  - Deploy and run arbitrary software, meaning middleware, and operating systems;
  - Control over operating systems, storage, and deployed applications
  - User does not manage or control the underlying Cloud infrastructure

Cloud Computing

- DBaaS (Cloud Computing)
  - Is an architectural and operational approach enabling DBAs to deliver database functionality as a service to internal and/or external users
  - Some IT specialists consider it as a part of PaaS, others don't

Cloud Computing

- DBaaS features:
  - Pre-configured, automatically installed database services for users
  - Management of database instances using on demand, self-service mechanisms to the users
  - Maximum availability

Cloud Computing

- PaaS (Cloud Computing):
  - Provides a platform allowing users to develop, run, and manage applications without the complexity of building and maintaining the infrastructure typically associated with developing and launching an application.
  - Is a complete development and deployment environment in the cloud, with resources that enable you to deliver everything from simple cloud-based apps to sophisticated, cloud-enabled enterprise applications

Cloud Computing

- PaaS features:
  - Deploy applications onto the Cloud infrastructure
  - Integrated development framework: programming languages, tools and libraries are supported by the provider
  - Analytics and or business intelligence tools are provided as a service with PaaS to allow users to analyze and mine their data
  - Control over the deployed applications

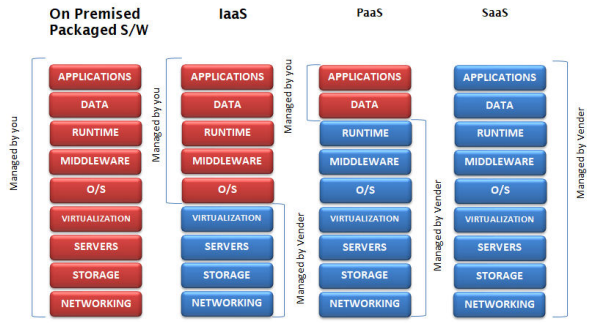
Cloud Computing

- SaaS (Cloud Computing):
  - Allows users to connect to and use cloud-based applications over the Internet
  - It is sometimes referred to as "on-demand software"

Cloud Computing

- SaaS features:
  - Applications are supplied by the cloud provider
  - Applications are accessible from various client devices (e.g. web browsers, APIs);
  - Examples: GMAIL, Dropbox, Office 365

Cloud Computing



Cloud Computing

- Deployment models for Cloud Computing:
  - Private cloud
  - Public cloud
  - Hybrid cloud



Cloud Computing

- Deployment models for Cloud Computing :
  - Private cloud
    - Single-tenant architecture
    - On-premise hardware
    - Direct control of infrastructure
    - E.g., OpenStrack, VMware ESXi, Kubernetes

Cloud Computing

- Deployment models for Cloud Computing :
  - Public cloud
    - Multi-tenant architecture
    - Pay-as-you-go pricing model
    - E.g., Amazon Web Services (AWS), Microsoft Azure, Google Cloud Platform

Cloud Computing

- Deployment models for Cloud Computing :
  - Hybrid cloud
    - Cloud bursting capabilities
    - Benefits both public and private environments
    - Combination of both public and private providers

Storage and tools

- Relational and NoSQL databases
- MapReduce
- Hadoop Ecosystem
- Spark Environment

Storage and tools

- Database (DB) is an organized collection of data. The collection of data is organized in schemas, tables, reports, views and other object.
- Database Management Systems (DBMS) is a computer software application that interacts with the user, other applications, and the database itself to capture and analyze data.
- A general-purpose DBMS is designed to allow the definition, creation, querying, update, and administration of databases.

Storage and tools

- There are multiple types of DBMS, usually classified by the way they store data:
  - Relational database management systems (RDBMSs): Oracle, Microsoft SQL Server, MySQL, PostgreSQL, IBM DB2, Hive
  - XML Databases: BaseX, eXist-db
  - Object-oriented DBMSs: ObjectDB, Caché
  - NoSQL DBMSs:
    - Key-Value DBMSs: **Riak**, Redis
    - Document-oriented DBMSs: **MongoDB**, Apache CouchDB
    - Column-oriented DBMSs: Apache HBase, **Cassandra**
    - Graph DBMSs: Neo4J

Storage and tools

- XML Databases
  - Stores the information in XML format
  - This data can be queried, transformed, exported and returned to a calling system
  - To manipulate the data, different technologies are used:
    - XQuery (XML Query) is a language used to query and transform collations of structured and unstructured data in XML format
    - XPath (XML Path Language) is a query language used for selecting nodes from a XML document
    - XSLT (Extensible Stylesheet Language Transformations) is a language for transforming XML documents into other XML documents or formats (e.g. HTML)

Storage and tools

- Object Oriented Databases:
  - Stores the information in the form of objects
  - Supports Object Oriented Programming principles: Polymorphism, Inheritance and Encapsulation
  - Are integrated to work with different frameworks, e.g., ObjectDB is integrated in Java EE (J2EE) and Spring web application and can be deployed on servlet containers (Tomcat, Jetty) as well as J2EE application servers (GlassFish, JBoss).
  - Support of different query languages, e.g., JDOQL (Java Data Objects Query language – based on the Java syntax), JPSQL (JPA Query Language), etc.

Storage and tools

- NoSQL Databases

Type	Example
Key-Value Store	redis, riak
Wide Column Store	HBASE, cassandra
Document Store	mongoDB, CouchDB
Graph Store	Neo4j, InfiniteGraph

Storage and tools

- NoSQL Databases Motives
  - Avoidance of unneeded complexity
    - Avoid strict data consistency
    - ACID properties for transactions are too restrictive
  - High throughput
    - NoSQL database provide a significantly higher data throughput than traditional RDBMSs
  - Easy horizontal scaling
  - Avoidance of expensive object-relational mapping
  - Decrease the complexity and cost of setting up database clusters
  - Compromising reliability for better performance
  - Moving to the cloud

Storage and tools

- NoSQL: Key-Value Databases
  - Stores the information as a map/dictionary
  - Allows clients to put and request values per key
  - Favor high scalability over consistency
  - Omit rich ad-hoc querying and analytics features (join and aggregate operations are set aside)

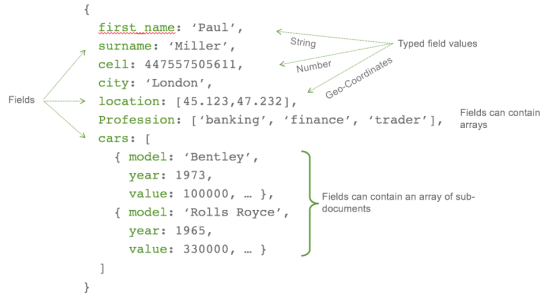
Key	Value
K1	AAA,BBB,CCC
K2	AAA,BBB
K3	AAA,DDD
K4	AAA,2,01/01/2015
K5	3,ZZZ,5623

Storage and tools

- NoSQL: Document-Oriented Databases
  - The next logical step from Key-Value Databases
  - Are schema-less semi-structured databases
  - Support datatypes and collections type (integers, floats, strings, datetime, array, objects, etc.)
  - Supports nested documents
  - MongoDB supports atomic update.

Storage and tools

- NoSQL: Document-Oriented Databases



Storage and tools

- NoSQL: Column-Oriented Databases
  - Are used to store and process data by column instead of row
  - Its origin is in analytics and business intelligence
  - Column-stores operate in a shared-nothing massively parallel processing architecture
  - Can be used to build high-performance applications

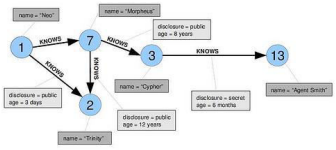
Storage and tools

- NoSQL: Column-Oriented Databases

KEY	COLUMN FAMILIES	
ID	CUSTOMERINFO	ADDRESSINFO
1001	FirstName: Tom MiddleName: T LastName: Tester	Address1: 2001 Bayfront Dr. Address2: Suite#813 City: Tampa State: FL Zip: 34637 Country: US
1002	FirstName: Bob MiddleName: B LastName: Builder	Address1: 1234 Sunny Circle City: Beverly Hills State: CA Zip: 90210

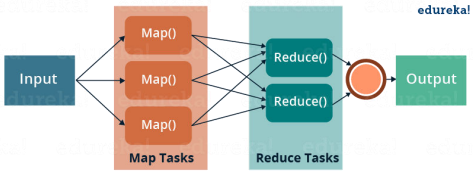
Storage and tools

- NoSQL: Graph Databases:
  - Uses graph structures for semantic queries with nodes, edges and properties to represent and store data
  - A key concept is the graph (vertices and edges) which directly relates data items in the store
    - The vertices are the items (data, things)
    - The edges represent the relationship between the data



Storage and tools

- MapReduce
  - Is a programming framework (paradigm or model) that allows us to perform distributed and parallel processing on large data sets in a distributed environment.

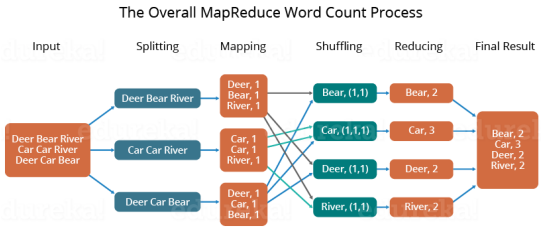


Storage and tools

- MapReduce
  - MapReduce consists of two distinct tasks – Map and Reduce
  - As the name MapReduce suggests, reducer phase takes place after mapper phase has been completed.
  - The output of a Mapper or map job (key-value pairs) is input to the Reducer.
  - The reducer receives the key-value pair from multiple map jobs.
  - Then, the reducer aggregates those intermediate data tuples (intermediate key-value pair) into a smaller set of tuples or key-value pairs which is the final output.

Storage and tools

- MapReduce word count example



Storage and tools

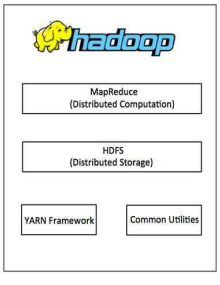
- Hadoop
  - Is a framework that allows for the distributed processing of large data sets across clusters of computers using simple programming models
  - Is designed to scale up from single servers to thousands of machines, each offering local computation and storage.
  - Rather than rely on hardware to deliver high-availability, the framework itself is designed to detect and handle failures at the application layer, so delivering a highly-available service on top of a cluster of computers, each of which may be prone to failures.

Storage and tools

- Hadoop includes these modules
  - Hadoop Common: The common utilities that support the other Hadoop modules
  - Hadoop Distributed File System (HDFS™): A distributed file system that provides high-throughput access to application data
  - Hadoop YARN: A framework for job scheduling and cluster resource management
  - Hadoop MapReduce: A YARN-based system for parallel processing of large data sets

Storage and tools

- Hadoop Ecosystem



Storage and tools

- HDFS
  - Hadoop distributed File System is based on Google File System (GFS)
  - Serves as the distributed file system for most tools in the Hadoop ecosystem
  - Scalability for large data sets
  - Reliability to cope with hardware failures

Storage and tools

- HDFS is good for:
  - Large files
  - Streaming data
- HDFS is not good for:
  - Lots of small files
  - Random access to files
  - Low latency access

Storage and tools

- Spark
  - An open-source and fast engine for large-scale data processing.
  - It supports data streaming and SQL, machine learning and graph processing.
  - Spark uses Hadoop's client libraries for HDFS and YARN

The NoSQL Movement

- The term NoSQL was first used in 1998 for relational database that omitted the use of SQL
- The term NoSQL was picked up again in 2009 by the advocates of non-relational databases
- The main reason for the NoSQL movement: to seek alternatives for solving problems for which relational databases are a bad fit

The NoSQL Movement Motives

1. Avoidance of unneeded complexity:
   - Relational databases provide a variety of features and strict data consistency
   - Relational databases use the ACID properties for transactions which are for some applications restrictive
   - The features, strict consistency and ACID properties of relational databases might be more than necessary for particular applications and use cases

The NoSQL Movement Motives

2. High throughput:
   - RDBMSs: Strict consistency and ACID properties restrict the throughput
   - NoSQL use looser consistency and different less restrictive transaction properties to improve throughput
   - NoSQL DBMSs use different storage methods than RDBMSs to improve throughput
   - NoSQL DBMSs' processing capabilities are improved using distributed and parallel algorithms, i.e. MapReduce

The NoSQL Movement Motives

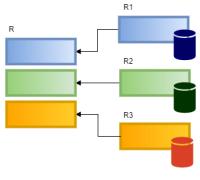
3. Horizontal scaling and running on commodity hardware:
   - Due to the huge volume of data, new technologies are needed to store and process this data
   - NoSQL DBMSs try to address the following current problems that DBMSs have:
     - Problems that appear when scaling out data due to the ACID properties
     - Performance of single servers
     - Processing of data on single servers
     - Rigid schema design
     - Costs of scaling, i.e. for Oracle, one needs at least Oracle Clusterware, Oracle Grid Infrastructure, and Oracle Real Application Server on top of which the database is installed

The NoSQL Movement Motives

3. Horizontal scaling and running on commodity hardware:
   - Fragmentation Horizontal (Sharding)

Fragmentarea pe orizontală.

În fragmentarea pe orizontală (Figura 4.1a), fragmentele R_i, $i = \overline{1,n}$, în care a fost împărțită relația R utilizează operația de reuniune pentru a reconstrui relația inițială, astfel $R = R_1 \cup R_2 \cup \ldots \cup R_n = \bigcup_{i=1}^{n} R_i$.

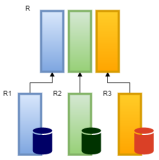


The NoSQL Movement Motives

3. Horizontal scaling and running on commodity hardware:
   - Fragmentation Vertical

Fragmentarea pe verticală.

În fragmentarea pe verticală (Figura 4.1b), fragmentele R_i, $i = \overline{1,n}$, în care a fost împărtită relația R utilizează operația de JOIN pentru a reconstrui relația inițială, astfel $R = R_1 \bowtie R_2 \bowtie \ldots \bowtie R_n$.

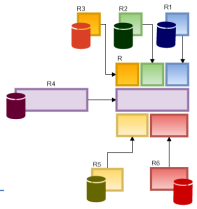


The NoSQL Movement Motives

3. Horizontal scaling and running on commodity hardware:
   - Fragmentation Mixt Horizontal-Vertical

Fragmentarea mixtă orizontală-verticală.

În fragmentare mixtă orizontală-verticală (Figura 4.2b), relația R este mai întâi fragmentată pe orizontală în relațiile R_i, $i = \overline{1,n}$, astfel încât $R = R_1 \cup R_2 \cup \ldots \cup R_n = \bigcup_{i=1}^{n} R_i$, și apoi fiecare relație R_i este la rândul ei fragmentată pe verticală în fragmentele R_{ij}, $j = \overline{1,m_i}$. Pentru a reconstruii fragmentele R_i se va utiliza operația de JOIN astfel: $R_i = R_{i1} \bowtie R_{i2} \bowtie \ldots \bowtie R_{im_i}$. Relația R este reconstruită utilizând mai întâi operația de JOIN și apoi cea de reuniunea: $R = (R_{11} \bowtie R_{12} \bowtie \ldots \bowtie R_{1m_1}) \cup (R_{21} \bowtie R_{22} \bowtie \ldots \bowtie R_{2m_2}) \cup \ldots \cup (R_{n1} \bowtie R_{n2} \bowtie \ldots \bowtie R_{nm_n})$.

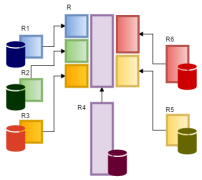


The NoSQL Movement Motives

3. Horizontal scaling and running on commodity hardware:
   - Fragmentation Mixt Vertical-Horizontal

Fragmentarea mixtă verticală-orizontală.

În fragmentare mixtă verticală-orizontală (Figura 4.2a), relația R este mai întâi fragmentată pe verticală în relațiile R_i, $i = \overline{1,n}$, astfel încât $R = R_1 \bowtie R_2 \bowtie \ldots \bowtie R_n$, și apoi fiecare relație R_i este la rândul ei fragmentată pe orizontală în fragmentele R_{ij}, $j = \overline{1,m_i}$. Pentru a reconstrui fragmentele R_i se va utiliza operația de reuniune astfel: $R_i = R_{i1} \cup R_{i2} \cup \ldots \cup R_{im_i}$. Relația R este reconstruită utilizând mai întâi operația de reuniune și apoi cea de JOIN: $R = (R_{11} \cup R_{12} \cup \ldots \cup R_{1m_1}) \bowtie (R_{21} \cup R_{22} \cup \ldots \cup R_{2m_2}) \bowtie \ldots \bowtie (R_{n1} \cup R_{n2} \cup \ldots \cup R_{nm_n})$.



The NoSQL Movement Motives

3. Horizontal scaling and running on commodity hardware:

- NoSQL DBMSs are designed to:
  - Easily scale out (horizontal scale)
    - Machines can be added and remove without causing the same operational efforts to perform sharding in RDBMS cluster-solutions
  - Not rely on highly available hardware
  - Flexible schema-less design
  - Lower costs of scaling

The NoSQL Movement Motives

4. Avoidance of Expensive object-relational mapping:

- Most NoSQL DBMSs are designed to store data that are either simple or more like the ones of object-oriented programing languages compared to relational data structures
- There is no need for expensive object-relational mapping
- This is particularly important for applications with data structures of low complexity that can hardly benefit from the features of a relational database

The NoSQL Movement Motives

5. Decrease the Complexity and the Cost of Setting up Database Clusters:

- NoSQL databases are designed in a way that PC clusters can be easily and cheaply expanded without the complexity and cost of sharding
- Sharding involves cutting up databases into multiple tables to run on large clusters or grids

The NoSQL Movement Motives

6. Compromising Reliability for Better Performance

- There are different scenarios for which applications can compromise reliability for better performance.
- E.g. HTTP session data which needs to be shared between various web servers but since the data is transient in nature (it goes away when the user logs off) there is no need to store it in persistent storage.

The NoSQL Movement Motives

7. The Current "One size fit's it all" Databases Thinking Was and Is Wrong

- A growing number of application scenarios cannot be addressed with a traditional database approach, e.g. Facebook, Twitter, Instagram, etc.
- The search for alternatives towards traditional RDBMSs can be explained by two major trends:
  - The continuous growth of data volumes (to be stored)
  - The growing need to process larger amounts of data in shorter time

The NoSQL Movement Motives

8. Movements in Programming Languages and Development Frameworks
   - Multiple programming languages and development frameworks provide abstractions for database access trying to hide the use of SQL and relational databases
   - The NoSQL databases react on this trend and try to provide data structures in their APIs that are closer to the ones of programming languages (e. g. key/value-structures, documents, graphs)
   - Examples for this trend in the last couple of years include:
     - Object-relational mappers in the Java and .NET world like the Java Persistence API (JPA, part of the EJB 3 specification), implemented by e.g., Hibernate, or the LINQ Framework with the ADO.NET Entity Framework since .NET version 4
     - The popularity of Python and Ruby frameworks which hide the usage of relational databases

The NoSQL Movement Motives

9. Requirements of Cloud Computing
   - There are two major requirements of datastores in cloud computing environments:
     - High until almost ultimate scalability (especially in the horizontal direction)
     - Low administration overhead
   - The following classes of databases work well in the cloud:
     - Data warehousing with specific features for batch data processing and map/reduce operations
     - Simple, scalable and fast key/value-stores
     - Databases containing a richer feature set than key/value-stores to fit the gap with traditional RDBMSs while offering good performance and scalability properties, e.g. document-oriented databases

NoSQL Databases Categories

- By scalability
- By data and query model
- By persistence design

NoSQL Databases Categories Scalability

- Through replication, it is easy to scale:
  - Read operations
  - Load distribution
- The write operations are important for the scalability due to data distribution and partitioning
- For distributed systems that provide auto sharding there are 2 important features:
  - Support of multiple datacenters
  - Possibility to add machines live to an existing cluster, transparent to the applications and the users using the cluster

NoSQL Databases Categories Data and query model

- Column-Oriented databases
- Key-Value databases
- Document-Oriented databases
- Graph databases

NoSQL Databases Categories Persistence design

1. In-Memory Databases
   - Are very fast
   - The data size is inherently limited by the size of RAM
   - Durability may become a problem as the amount of data which can get lost between subsequent disk flushes is potentially large
     - E.g. due to server crashes or by losing power supply

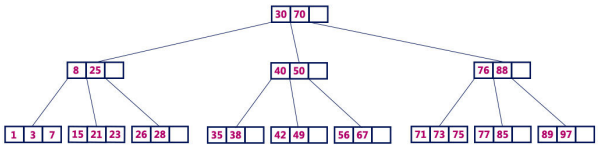
NoSQL Databases Categories Persistence design

2. Memtables and SSTables:
   - Write operations are buffered in memory (in a Memtable) after they have been written to an append-only commit log to ensure durability
   - After a certain amount of writes the Memtable gets flushed to disk (as a whole) to a SSTable (Idea taken from Google's Bigtable paper)
   - SSTable persistence strategy has performance characteristics comparable to those of in-memory-databases but avoids the durability difficulties of pure in-memory databases
   - Compared to disk-based strategies, disk seeks are reduced due to the append-only log and the flushing of whole Memtables to disk

NoSQL Databases Categories Persistence design

3. B-trees
   - Have been used in databases since their beginning to provide a robust indexing-support
   - Their performance characteristics on rotating disks are not very positive due to the amount of disk seeks needed for read and write operations
   - The document-oriented database CouchDB uses B-trees internally but tries to avoid the overhead of disk seeks by only appending to B-trees (append-only B-trees):
     - Only one write operation at a time can happen
     - Concurrent reads to different sections of a B-tree are not allowed in this case

NoSQL Databases Categories
Persistence design

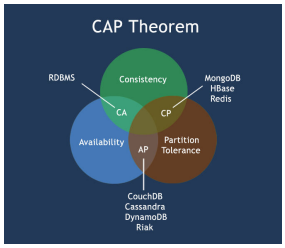


NoSQL Databases Categories

Type	Performance	Scalability	Flexibility	Complexity	Functionality
Key-Value	High	High	High	None	Variable
Column	High	High	Moderate	Low	Minimal
Document	High	Variable (high)	High	Low	Variable
Graph	Variable	Variable	High	High	Graph theory
Relational	Variable	Variable	Low	Moderate	Relational algebra

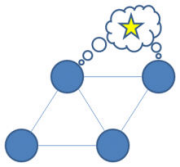
CAP Theorem

- Consistency
- Availability
- Partition Tolerance



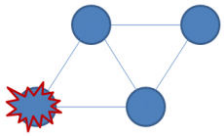
CAP Theorem
Consistency

- Consistency meaning if and how a system is in a consistent state after the execution of an operation
- A distributed system is typically considered to be consistent if after an update operation of some writer all readers see his updates in some shared data source



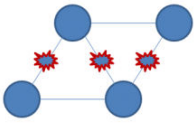
CAP Theorem
Availability

- Availability and especially high availability meaning that a system is designed and implemented in a way that allows it to continue operation (i.e. allowing read and write operations) even if nodes in a cluster crash or some hardware or software parts are down due to upgrades.



CAP Theorem
Partition Tolerance

- Partition Tolerance is understood as the ability of the system to continue operation in the presence of network partitions
- These occur if two or more "islands" of network nodes arise which (temporarily or permanently) cannot connect to each other.
- Some people also understand partition tolerance as the ability of a system to cope with the dynamic addition and removal of nodes (e. g. for maintenance purposes; removed and again added nodes are considered an own network partition in this notion).



CAP Theorem

- Cap Theorem (Pick 2)

	Traits	Examples
Consistence + Availability	• 2-phase commit (2PC) • Cache-validation protocols	RDBMS: Oracle, MySQL, MS SQL Server, PostgreSQL
Consistence + Partition Tolerance	• Pessimistic locking • Make minority partitions unavailable	MongoDB, HBase, Redis
Availability + Partition Tolerance	• Conflict resolution • Optimistic locking	CouchDB, Cassandra, Riak

CAP Theorem
Consistence + Availability

- 2-phase commit
  - A type of atomic commitment protocol
  - A distributed algorithm that coordinates all the processes that participate in a distributed atomic transaction on whether to commit or abort (roll back) the transaction
- Cache validation protocols:
  - Ensures cache coherence
  - Cache coherence is the uniformity of shared resource data that ends up stored in multiple local caches

CAP Theorem

- Minority partition (Consistence + Partition Tolerance)
  - A partition of nodes fewer or equal than half of the total number of nodes
  - Partions (islands) appear during communition failure between the nodes in a cluster
- Conflict resolution (Availability + Partition Tolerance)
  - Conflicts can occur in an environment that permits concurrent updates to the same data at multiple sites
  - Solutions: Multiversion concurrency control

CAP Theorem

- Pessimistic locking (Consistence + Partition Tolerance)
  - A resource
    - Is locked from the time it is first accessed in a transaction until the transaction is finished
    - Is inaccessible to other transactions during that time.
  - If most transactions simply look at the resource and never change it
    - An exclusive lock may be overkill as it may cause lock contention
    - Optimistic locking may be a better approach
  - Locks are applied in a fail-safe way
  - The lock exists until the transaction has either been committed or rolled back

CAP Theorem

- Optimistic locking (Availability + Partition Tolerance)
  - A resource is not actually locked when it is first accessed by a transaction
    - Other transactions can concurrently access the resource
    - Conflicting changes are possible
  - At commit time, when the resource is about to be updated in persistent storage the state of the resource
    - Is read from storage again
    - Is compared to the state that was saved when the resource was first accessed in the transaction
    - If the two states differ, a conflicting update was made, and the transaction will be rolled back

Transaction properties ACID

- ACID (**A**tomicity, **C**onsistency, **I**solation, **D**urability)
  - **Atomicity**:
    - Requires that each transaction be "all or nothing"
    - If one part of the transaction fails, then the entire transaction fails, and the database state is left unchanged.
  - **Consistency** ensures that any transaction will bring the database from one valid state to another.
  - **Isolation** ensures that the concurrent execution of transactions results in a system state that would be obtained if transactions were executed sequentially, i.e., one after the other.
  - **Durability** ensures that once a transaction has been committed, it will remain so, even in the event of power loss, crashes, or errors.

Transaction properties BASE

- BASE (**B**asically **A**vailable, **S**oft-sate, **E**ventual consistency)
- **Basically Available:**
  - This constraint states that the system does guarantee the availability of the data as defined by the CAP Theorem.
  - There will be a response to any request
  - The response could still be 'failure' to obtain the requested data, or the data may be in an inconsistent or changing state
- **Soft-state:**
  - The state of the system could change over time, so even during times without input there may be changes going on due to 'eventual consistency', thus the state of the system is always 'soft'
- **Eventual consistency:**
  - The system will eventually become consistent once it stops receiving input.
  - The data will propagate to everywhere it should sooner or later, but the system will continue to receive input and is not checking the consistency of every transaction before it moves onto the next one

ACID vs BASE

ACID	BASE
Strong consistency	Weak consistency
Isolation	Availability first
Focus on “commit”	Best effort
Real answers	Approximate answers
Availability	Aggressive Availability
Difficulty in modeling	Easier modeling

Key-Value Databases

- Have a simple data model: map/dictionary
- Allows clients to request values by key
- Favor high availability over consistency
- Simple queries, they omit rich ad-hoc querying and analytics features – joins and aggregations are set aside
- The length of the keys are limited by a certain number of bytes while there is less limitation in values
- A great influence on the architecture is Amazon Dynamo Database

Riak

- Is a distributed key-value store
- Values can be anything from pain text, JSON, or XML to Images of video clips
- All the values are accessible through a simple HTTP interface
- Is fault tolerant – nodes can go up and down at any moment without any single point of failure
- A Riak cluster works continuously as nodes are added, removed or (ideally not) crush

Riak

- Riak trade-offs:
  - Lacks robust support for ad-hoc queries
  - Trouble linking values together (no foreign keys)

Riak

- Loves the web:
  - Querying is done via URLs, headers, and verbs
  - Riak returns assets and standard HTTP response codes
- Riak supplies an HTTP REST interface
- The interaction is done via the URL tool cURL
- cURL allows the interaction with the REST API without resorting to a particular driver or programming language

Riak

- Riak is a great choice for datacenters that must serve many requests with low latency
- It's easy to manage, easy to set up, and can grow with your needs
- It is built in Erlang
- Can be installed on different distributions of Linux and Mac OS

Riak

- Riak's default mode of operation is to work as a cluster consisting of multiple nodes, i.e. multiple well-connected data hosts
- Each host in the cluster runs a single instance of Riak, referred to as a Riak node
- Each Riak node manages a set of virtual nodes (vnodes) that are responsible for storing a separate portion of the keys stored in the cluster

Riak

- Riak nodes (in contrast to some high-availability systems)
  - Are not clones of one another
  - Do not all participate in fulfilling every request
- At runtime or at request time, you can configure:
  - The number of nodes on which data is to be replicated
  - When replication occurs
  - Which merge strategy and failure model are to be followed.

Riak

- Any client interface to Riak interacts with objects in terms of the bucket and key in which a value is stored (as well as the bucket type that is used to set the bucket's properties)
- Internally,
  - Riak computes a 160-bit binary hash of each bucket/key pair
  - Maps the hash value to a position on an ordered ring of all such values
- This ring is divided into partitions
- Each Riak **vnode** responsible for one of these partitions (meaning that each **vnode** claims that partition)

Riak

- The nodes of a Riak cluster each attempt to run a roughly equal number of **vnodes** at any given time
- In the general case, this means that each node in the cluster is responsible for
  - 1/(number of nodes) of the ring
  - (number of partitions)/(number of nodes) vnodes
- E.g., If two nodes define a 16-partition cluster then each node will run 8 vnodes
- Nodes attempt to claim their partitions at intervals around the ring such that
  - There is an even distribution amongst the member nodes
  - No node is responsible for more than one replica of a key

Riak
Intelligent Replication

- When an object is being stored in the cluster, any node may participate as the coordinating node for the request
- The coordinating node
  - Consults the ring state to determine which vnode owns the partition in which the value's key belongs
  - Sends the write request to that vnode as well as to the vnodes responsible for the next N-1 partitions in the ring (where N is a configurable parameter that describes how many copies of the value to store)

Riak
Intelligent Replication

- The write request may also specify that
  - At least W (=< N) of those vnodes reply with success
  - DW (=< W) nodes reply with success only after durably storing the value
- A read, or GET, request operates similarly, sending requests
  - To the vnode that "claims" the partition in which the key resides
  - As well as to the next N-1 partitions
- The request also specifies R (=< N), the number of vnodes that must reply before a response is returned

Riak
Intelligent Replication

- When N is set to 3, the value REM is stored in the key artist
- That key is assigned to 3 partitions out of 32 available partitions
- When a read request is made to Riak, the ring state will be used to determine which partitions are responsible
- From there, a variety of configurable parameters determine how Riak will behave in case the value is not immediately found

Intelligent Replication
Gossiping

- The ring state is shared around the cluster by means of a "gossip protocol"
- Whenever a node changes its claim on the ring, it announces, i.e. "gossips", this change to other nodes so that the other nodes can respond appropriately
- Nodes also periodically re-announce what they know about ring in case any nodes happened to miss previous updates

Installing Riak

- Installing Riak
  - Go to the official web site http://basho.com/products/ and download the package for your distribution.
  - You can also compile the source code but you need to install Erlang and additional packages.

Installing Riak

- Installing Riak on Ubuntu 22.04:
  - Download the latest version:
    wget https://files.tiot.jp/riak/kv/3.2/3.2.0/ubuntu/jammy64/riak_3.2.0-OTP25_amd64.deb
  - Install the package
    sudo dpkg -i riak_3.2.0-OTP25_amd64.deb
  - Start a node on localhost (wither with sudo privileges or as root)
    sudo riak daemon

Installing Riak on a cluster

- Download and install the package on all the nodes - do not start Riak
- On each node modify riak.conf file (the default location is /etc/riak/riak.conf)
- Give a name to the Erlang node (nodename)
  nodename = riak@ip_host
- Set the IP Address and TCP port to bind Riak's HTTP interface
  listener.http.internal = ip_host:port
- Set the IP Address and TCP port to bind Riak's Protocol Buffers interface (Protocol Buffers is a method to serialize data)
  listener.protobuf.internal = ip_host:port

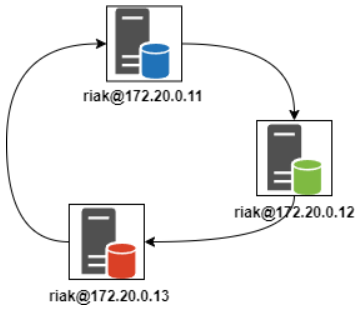
Installing Riak on a cluster

- Enable VM Tunnings
  erlang.schedulers.force_wakeup_interval = 500
  erlang.schedulers.compaction_of_load = false
- Set the ring size
  ring_size = 64
- Start the node
  $ sudo riak daemon

Installing Riak on a cluster

- If the master node is now properly set up proceed to adding other nodes to the cluster by doing the same configurations as for the master
- The new nodes must join the cluster, on each node run the following command
  $ sudo riak admin cluster join riak@ip_master
- Next, plan and commit the changes on the master node:
  $ sudo riak admin cluster plan
  $ sudo riak admin cluster commit

Riak Cluster Example



Riak Cluster Example

- On node riak@172.20.0.11 edit riak.conf as follows:
  nodename = riak@172.20.0.11
  listener.http.internal = 172.20.0.11:8098
  listener.protobuf.internal = 172.20.0.11:8097
  ring_size = 64
  erlang.schedulers.force_wakeup_interval = 500
  erlang.schedulers.compaction_of_load = false
- Start the node (as root):
  $ riak daemon

Riak Cluster Example

- On node riak@172.20.0.12 edit riak.conf as follows:
  nodename = riak@172.20.0.12
  listener.http.internal = 172.20.0.12:8098
  listener.protobuf.internal = 172.20.0.12:8097
  ring_size = 64
  erlang.schedulers.force_wakeup_interval = 500
  erlang.schedulers.compaction_of_load = false
- Start the node (as root):
  $ riak daemon

Riak Cluster Example

- On node riak@172.20.0.13 edit riak.conf and add as follows:
  nodename = riak@172.20.0.13
  listener.http.internal = 172.20.0.13:8098
  listener.protobuf.internal = 172.20.0.13:8097
  ring_size = 64
  erlang.schedulers.force_wakeup_interval = 500
  erlang.schedulers.compaction_of_load = false
- Start the node (as root):
  $ riak daemon

Installing Riak on a cluster

- Run on nodes riak@172.20.0.12 and riak@172.20.0.13 the following command (as root):
  $ riak admin cluster join riak@172.20.0.11
- Run on riak@172.20.0.11 the following command (as root):
  $ riak admin cluster plan
  $ riak admin cluster commit

Installing Riak on a cluster

- The command line administrator tool is: riak admin
- To check the cluster status
  $ sudo riak admin cluster status

```
root@770d7a85290e:/# sudo riak admin cluster status
---- Cluster Status ----
Ring ready: true

+----------------------+------+-------+-----+-------+
|         node         |status| avail |ring |pending|
+----------------------+------+-------+-----+-------+
| (C) riak@172.20.0.11 |valid |  up   | 34.4|  --   |
|     riak@172.20.0.12 |valid |  up   | 32.8|  --   |
|     riak@172.20.0.13 |valid |  up   | 32.8|  --   |
+----------------------+------+-------+-----+-------+

Key: (C) = Claimant; availability marked with '!' is unexpected
ok
```


Installing Riak on a cluster

- To see partitions assigned for each node
  $ sudo riak admin cluster partitions

```
root@770d7a85290e:/# sudo riak admin cluster partitions
Partitions owned by 'riak@172.20.0.11':
+---------+----------------------------------------+---+
|  type   |                 index                  |(a)|
+---------+----------------------------------------+---+
| primary |                   0                    | 0 |
| primary | 913438523331814323877388204476768872849578393 6 |
| primary |1826877004666362864775460604089535377456991567872 8 |
| primary |2740315569995U4297161190961343030661858873518088 12|
| primary |3653754093127257293509212081790797594913983135744 16|
| primary |4567192616659071619386515102238384N3642478919680 20|
| primary |5480631139990885994326381012268606132370974703616 24|
| primary |6394086966322279892671411211413137382189478487552 28|
| primary |7307508186664514591818422416358141509827966271488 32|
| primary |8220946789986328914895727184829091985564620554424 36|
| primary |9134385233318143230773030284476768872849578393968 40|
| primary |9813946412581708398168108744981252653831329677312 43|
| primary |1004782375664995756265833224924445760134536233964 44|
| primary |1073299264091488183855581104926203422559825461248 47|
| primary |1096126227998177188652766249537212264741949407232 48|
| primary |1146463417248066124294356135107078831283121265184 51|
| primary |1187070080331358621840803926581979953479848519168 52|
| primary |1255977969581240469533129165311555572081681702912O 55|
| primary |1278813932664504085342822062867076621989040975104 56|
| primary |1347321821914042612797190219551603234087453128130564 59|
| primary |1370157784997721485819595453867151533892743675904O 60|
| primary |1438666567424768756810675225726509189747388859609 2 63|
|secondary|                   --                   |---|
| stopped |                   --                   |---|
+---------+----------------------------------------+---+

ok
```


Riak Statistics

- To see the statistics of the server
  $ sudo riak admin status
- cURL can be used to get the statistics:
  $ curl -GET http://172.20.0.11:8098/stats
- To check Riak configuration
  $ sudo riak chkconfig
- Ping the cluster to see if it's up and running:
  $ sudo riak chkconfig
  $ curl -GET http://172.20.0.11:8098/ping

Riak and cURL

- REST stands for REpresentational State Transfer
- REST is a guideline for mapping resources to URLs and interacting with them using CRUD verbs:
  - POST (**C**reate)
  - GET (**R**ead)
  - PUT (**U**pdate)
  - DELETE (**D**elete).
- cURL is used to directly connect to Riak server's HTTP REST interface without the need for an interactive console

Riak and cURL

- If we want only the header response use –I

$ curl -I http://172.20.0.11:8098/riak/animals

```
root@770d7a85290e:/# curl -I http://172.20.0.11:8098/riak/animals
HTTP/1.1 200 OK
Vary: Accept-Encoding
Server: MochiWeb/3.0.0 WebMachine/1.11.1 (greased slide to failure)
Date: Mon, 27 Nov 2023 17:57:47 GMT
Content-Type: application/json
Content-Length: 520
```


Riak and cURL PUT

- Adding a new key-value element
  - Use curl with -X PUT or -XPUT
  - The -H attribute sets the following text as HTTP header information. In the example, we set the MIME content type to HTML.
  - Everything passed to -d (also known as the body data) is what Riak will add as a new value.
  - The -v attribute stands for verbose
  - The bucket is favs (equivalent to a schema)
  - The key is db

```
$ curl -v -X PUT http://172.20.0.11:8098/riak/favs/db \
-H "Content-Type: text/html" \
-d "<html><body><h1>My new favorite DB is RIAK</h1></body></html>"
```


Riak and cURL PUT

- Adding a new key-value element
  - Use curl with -X PUT or -XPUT
  - The -H attribute sets the following text as HTTP header information. In the example, we set the MIME content type to HTML.
  - Everything passed to -d (also known as the body data) is what Riak will add as a new value.
  - The -v attribute stands for verbose
  - The bucket is favs (equivalent to a schema)
  - The key is db

```
$ curl -v -X PUT http://172.20.0.11:8098/riak/favs/db \
-H "Content-Type: text/html" \
-d "<html><body><h1>My new favorite DB is RIAK</h1></body></html>"
```


Riak and cURL POST

- The attribute POST is used if we want the system to generate a new key by itself

```
curl -v -X POST http://172.20.0.11:8098/riak/favs \
-H "Content-Type: text/html" \
-d "<html><body><h1>Auto generated key</h1></body></html>"
```


Riak and cURL POST

```
root@770d7a85290e:/# curl -v -X POST http://172.20.0.11:8098/riak/favs \
-H "Content-Type: text/html" \
-d "<html><body><h1>Auto generated key</h1></body></html>"
Note: Unnecessary use of -X or --request, POST is already inferred.
*   Trying 172.20.0.11:8098...
* Connected to 172.20.0.11 (172.20.0.11) port 8098 (#0)
> POST /riak/favs HTTP/1.1
> Host: 172.20.0.11:8098
> User-Agent: curl/7.81.0
> Accept: */*
> Content-Type: text/html
> Content-Length: 53
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 201 Created
< Vary: Accept-Encoding
< Server: MochiWeb/3.0.0 WebMachine/1.11.1 (greased slide to failure)
< Location: /riak/favs/MllwGkO6FvVSefrED1asOeq40TW
< Date: Mon, 27 Nov 2023 18:03:59 GMT
< Content-Type: text/html
< Content-Length: 0
<
* Connection #0 to host 172.20.0.11 left intact
```

New Key

Riak and cURL DELETE

- The DELETE attribute is used to delete a record

$ curl -v -X DELETE http://172.20.0.11:8098/riak/favs/MllwGkO6FvVSefrED1asOeq40TW

```
root@770d7a85290e:/# curl -v -X DELETE http://172.20.0.11:8098/riak/favs/MllwGkO6FvVSefrED1asOeq40TW
*   Trying 172.20.0.11:8098...
* Connected to 172.20.0.11 (172.20.0.11) port 8098 (#0)
> DELETE /riak/favs/MllwGkO6FvVSefrED1asOeq40TW HTTP/1.1
> Host: 172.20.0.11:8098
> User-Agent: curl/7.81.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 204 No Content
< Vary: Accept-Encoding
< Server: MochiWeb/3.0.0 WebMachine/1.11.1 (greased slide to failure)
< Date: Mon, 27 Nov 2023 18:06:03 GMT
< Content-Type: text/html
< Content-Length: 0
<
* Connection #0 to host 172.20.0.11 left intact
```


Riak and cURL GET

- The GET attribute is used to get the information for a record

$ curl -v -X GET http://172.20.0.11:8098/riak/favs/db

```
root@770d7a85290e:/# curl -v -X GET http://172.20.0.11:8098/riak/favs/db
Note: Unnecessary use of -X or --request, GET is already inferred.
*   Trying 172.20.0.11:8098...
* Connected to 172.20.0.11 (172.20.0.11) port 8098 (#0)
> GET /riak/favs/db HTTP/1.1
> Host: 172.20.0.11:8098
> User-Agent: curl/7.81.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< X-Riak-Vclock: a85hYGBgzGDKBVI8qUcPb1ytZ6IDEUpkzGNleB1y8x5fFgA=
< Vary: Accept-Encoding
< Server: MochiWeb/3.0.0 WebMachine/1.11.1 (greased slide to failure)
< Link: </riak/favs>; rel="up"
< Last-Modified: Mon, 27 Nov 2023 17:59:07 GMT
< ETag: "2ucL3c2ubdgXI1enKHu0X4"
< Date: Mon, 27 Nov 2023 18:07:00 GMT
< Content-Type: text/html
< Content-Length: 61
<
* Connection #0 to host 172.20.0.11 left intact
<html><body><h1>My new favorite DB is RIAK</h1></body></html>
```

Riak Example

- Example: Animal shelter
- Let's create a shelter for animals and add 2 dogs

```
curl -v -X PUT http://172.20.0.11:8098/riak/animals/ace \
-H "Content-Type: application/json" \
-d '{"nickname" : "The Wonder Dog", "breed" : "German Shepherd"}'

curl -v -X PUT http://172.20.0.11:8098/riak/animals/polly?returnbody=true \
-H "Content-Type: application/json" \
-d '{"nickname" : "Sweet Polly Purebred", "breed" : "Purebred"}'
```


Riak links

- Links are metadata that associate one key to other keys
- The basic structure is this:

Link: </riak/bucket/key>; riaktag=\"whatever\"

- The key to where this value links is in pointy brackets (<…>), followed by a semicolon and then a tag describing how the link relates to this value (it can be whatever string we like).
- There can be many metadata links as necessary separated by commas
- Links are one-directional

Riak Example

- Let's add 2 rooms for the dogs with links

```
curl -X PUT http://172.20.0.11:8098/riak/cages/1 \
-H "Content-Type: application/json" \
-H "Link: </riak/animals/polly>; riaktag=\"contains\"" \
-d '{"room" : 101}'

curl -X PUT http://172.20.0.11:8098/riak/cages/2 \
-H "Content-Type: application/json" \
-H "Link:</riak/animals/ace>;riaktag=\"contains\", </riak/cages/1>;riaktag=\"next_to\"" \
-d '{"room" : 102}'
```


Riak Example

- The links are seen only on the room records, not the animal records

```
curl -i http://172.20.0.11:8098/riak/animals/polly
curl -i http://172.20.0.11:8098/riak/animals/ace

curl -i http://172.20.0.11:8098/riak/cages/1
curl -i http://172.20.0.11:8098/riak/cages/2
```


Riak Example

```
ciprian@hadoop-master:~$ curl -i -w '\n' http://172.19.3.36:8098/riak/animals/ace
HTTP/1.1 200 OK
X-Riak-Vclock: a85hYGBgymDKBVI8WQtdOxbpPrRlYFAXymBKZMpjZXgiOXaZDyp9aoWIObKdGjxAaUGo9EtVoHQWAA==
Vary: Accept-Encoding
Server: MochiWeb/1.1 WebMachine/1.10.9 (cafe not found)
Link: </riak/animals>; rel="up"
Last-Modified: Sun, 19 Nov 2017 15:57:29 GMT
ETag: "4J49QrCprFctfUZOlNwZb"
Date: Sun, 19 Nov 2017 16:05:02 GMT
Content-Type: application/json
Content-Length: 60

{"nickname" : "The Wonder Dog", "breed" : "German Shepherd"}
ciprian@hadoop-master:~$ curl -i -w '\n' http://172.19.3.36:8098/riak/cages/2
HTTP/1.1 200 OK
X-Riak-Vclock: a85hYGBgzGDKBVI8Z49eOC65smo/A4OfcgZTInMeK8NX1bbLfFkA
Vary: Accept-Encoding
Server: MochiWeb/1.1 WebMachine/1.10.9 (cafe not found)
Link: </riak/animals/ace>; riaktag="contains", </riak/cages/1>; riaktag="next_to", </riak/cages>; rel="up"
Last-Modified: Sun, 19 Nov 2017 15:57:41 GMT
ETag: "4VP2pyDp4OF0UwpULVTW8A"
Date: Sun, 19 Nov 2017 16:05:08 GMT
Content-Type: application/json
Content-Length: 14

{"room" : 102}
```


Riak Link Walking

- Getting the linked data is achieved by appending a link spec to the URL that is structured like this: /_,_,_
- The underscores (_) in the URL represent wildcards to each of the link criteria:
  - Bucket
  - Tag (name of link)
  - Keep
- Keep (1 or 0) is used to keep the link when traversing multiple links
- It returns a multipart/mixed dump of headers plus bodies of all linked keys/values.
- Replace the underscores in the link spec to filter only values we want

Riak Example

```
curl http://172.20.0.11:8098/riak/cages/1/_,_,_

curl http://172.20.0.11:8098/riak/cages/1/_,contains,_

curl http://172.20.0.11:8098/riak/cages/2/_,contains,_

curl http://172.20.0.11:8098/riak/cages/2/_,next_to,_

curl http://172.20.0.11:8098/riak/cages/2/cages,next_to,1/animals,_,_

curl http://172.20.0.11:8098/riak/cages/2/cages,next_to,0/animals,_,_
```


Riak MapReduce

